

Problem 2

[10p] Show that the recursive version of DELETE algorithm for AVL trees runs in $O(\log n)$.

Hint: you need to show the complexity of all functions that are invoked by DELETE.

We know the complexity = (complexity per level) $\times$ (height of tree) for a recursive method that go left or right in one level

Let's first consider the worst case and assume the total number of nodes is n

Then the height of the AVL tree is $\log n$

```
1      //O(log n)
2      private AVLNode delete(AVLNode curr, int val) {
3          //Case1: This case is O(1)
4          if (curr == null) {
5              return null; //O(1)
6          }
7          /* Case 2: at this level, the work is O(1) ,
8             * then we go down one level by calling delete on curr.left
9             * In worst case, we can go down (the height of the tree) levels,
10            * so the total work in this path is O(1) * log (n) = O(log n).
11            */
12         else if (val < curr.val) {
13             curr.left = delete(curr.left, val);
14         }
15         /* Case 3: at this level, the work is O(1),
16            * then we go down one level by calling delete on curr.right
17            * In worst case, we again go down (the height of the tree) levels,
18            * so the total work in this path is O(1) * log (n) = O(log n).
19            */
20         else if (val > curr.val) {
21             curr.right = delete(curr.right, val);
22         }
23         //Case 4:
24         else {
25             //Subcase 1: O(1)
26             if (curr.left == null) {
27                 return curr.right; //O(1)
28             }
29             //Subcase 2: O(1)
30             else if (curr.right == null) {
31                 return curr.left; //O(1)
32             }
33             //Subcase 3:
34             else {
35                 //O(log n) since findSuccessor is O(log n). This method will be called
at most one time per delete!!!
36                 AVLNode succ = findSuccessor(curr);
37                 //O(1)
38                 curr.val = succ.val;
```

```

39         /* For this recursive method, the work is  $O(1)$  per level,
40         * then go down and the height of the tree starting
41         * at curr.right is less or equal to  $\log(n)$ .
42         * Thus in the worst case we need go through (the height of the tree)
levels,
43         * Thus complexity is  $O(1) * \log(n) = O(\log n)$ .
44         */
45         curr.right = delete(curr.right, succ.val);
46         //Thus in case 4, the complexity is also  $O(\log n)$  not  $\log(\log^2 n)$  since
`findSuccessor` is called at most one time
47     }
48 }
49 //  $O(1)$  since rebalance is  $O(1)$  and rebalance is called once per node
50 return rebalance(curr);
51 }
52
53 //Worst case:  $O(\log n)$ 
54 private AVLNode findSuccessor(AVLNode curr) {
55     // $O(1)$ 
56     AVLNode succ = curr.right;
57     //  $O(1) * \log n = O(\log n)$  when curr is root
58     while (succ.left != null) {
59         succ = succ.left;
60     }
61     return succ;
62 }
63
64 //  $O(1)$ 
65 private AVLNode rebalance(AVLNode curr) {
66     //  $O(1)$  since balanceFactor, height, rightRotate and leftRightRotate are all
 $O(1)$ 
67     if (balanceFactor(curr) < -1) {
68         if(height(curr.left) >= height(curr.right))
69             curr = rightRotate(curr);
70         else
71             curr = leftRightRotate(curr);
72     }
73     //  $O(1)$  since balanceFactor, height, leftRotate and rightLeftRotate are all  $O(1)$ 
74     else if (balanceFactor(curr) > 1) {
75         if(height(curr.right) >= height(curr.left))
76             curr = leftRotate(curr);
77         else
78             curr = rightLeftRotate(curr);
79     }
80     // $O(1)$  since updateHeight is  $O(1)$ 
81     updateHeight(curr);
82     return curr;
83 }
84
85 //  $O(1)$ 
86 private AVLNode leftRotate(AVLNode p) {
87     // $O(1)$ 

```

```

88     AVLNode q = p.right;
89     p.right = q.left;
90     if(p == root)
91         root = q;
92
93     // O(1) since updateParentsLeftRotate is O(1)
94     updateParentsLeftRotate(p, q);
95     q.left = p;
96     // O(1) since updateHeight is O(1)
97     updateHeight(p);
98     updateHeight(q);
99     return q;
100 }
101
102 //O(1)
103 private void updateParentsLeftRotate(AVLNode p, AVLNode q) {
104     //O(1)
105     if (q.left != null) {
106         q.left.parent = p;
107     }
108     //O(1)
109     q.parent = p.parent;
110     //O(1)
111     if (p.parent != null) {
112         if (p == p.parent.left) {
113             p.parent.left = q;
114         } else {
115             p.parent.right = q;
116         }
117     }
118     //O(1)
119     p.parent = q;
120
121 }
122
123 // O(1)
124 private AVLNode rightRotate(AVLNode p) {
125     //O(1)
126     AVLNode q = p.left;
127     p.left = q.right;
128     if(p == root)
129         root = q;
130
131     // O(1)
132     updateParentsRightRotate(p, q);
133     //O(1)
134     q.right = p;
135
136     //O(1) since updateHeight is O(1)
137     updateHeight(p);
138     updateHeight(q);
139     return q;

```

```

140     }
141
142     //O(1)
143     private void updateParentsRightRotate(AVLNode p, AVLNode q) {
144         //O(1)
145         if (q.right != null) {
146             q.right.parent = p;
147         }
148         //O(1)
149         q.parent = p.parent;
150         //O(1)
151         if (p.parent != null) {
152             if (p == p.parent.right) {
153                 p.parent.right = q;
154             } else {
155                 p.parent.left = q;
156             }
157         }
158         //O(1)
159         p.parent = q;
160
161     }
162
163     // O(1) since leftRotate and rightRotate are both O(1)
164     private AVLNode leftRightRotate(AVLNode curr) {
165         curr.left = leftRotate(curr.left);
166         return rightRotate(curr);
167     }
168
169     // O(1) since leftRotate and rightRotate are both O(1)
170     private AVLNode rightLeftRotate(AVLNode curr) {
171         curr.right = rightRotate(curr.right);
172         return leftRotate(curr);
173     }
174
175     //O(1) since height is O(1)
176     public int balanceFactor(AVLNode curr) {
177         return height(curr.right) - height(curr.left);
178     }
179
180     //O(1)
181     public int height() {
182         return height(root);
183     }
184
185     //O(1)
186     private int height(AVLNode curr) {
187         if (curr == null) {
188             return 0;
189         } else {
190             return curr.height;
191         }

```

```
192     }
193
194     // O(1)
195     private void updateHeight(AVLNode curr) {
196         curr.height = 1 + Math.max(height(curr.left), height(curr.right));
197     }
```